

RandomForestRegressor

```
class sklearn.ensemble.RandomForestRegressor(n_estimators=100, *,
criterion='squared_error', max_depth=None, min_samples_split=2,
min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=1.0,
max_leaf_nodes=None, min_impurity_decrease=0.0, bootstrap=True, oob_score=False,
n_jobs=None, random_state=None, verbose=0, warm_start=False, ccp_alpha=0.0,
max_samples=None, monotonic_cst=None) # \[source\]
```

A random forest regressor.

A random forest is a meta estimator that fits a number of decision tree regressors on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control overfitting. Trees in the forest use the best split strategy, i.e. equivalent to passing `splitter="best"` to the underlying [DecisionTreeRegressor](#). The sub-sample size is controlled with the `max_samples` parameter if `bootstrap=True` (default), otherwise the whole dataset is used to build each tree.

This estimator has native support for missing values (NaNs). During training, the tree grower learns at each split point whether samples with missing values should go to the left or right child, based on the potential gain. When predicting, samples with missing values are assigned to the left or right child consequently. If no missing values were encountered for a given feature during training, then samples with missing values are mapped to whichever child has the most samples.

For a comparison between tree-based ensemble models see the example [Comparing Random Forests and Histogram Gradient Boosting models](#).

Read more in the [User Guide](#).

Parameters:

n_estimators : *int, default=100*

The number of trees in the forest.

 **Changed in version 0.22:** The default value of `n_estimators` changed from 10 to 100 in 0.22.

criterion : *{“squared_error”, “absolute_error”, “friedman_mse”, “poisson”}, default=“squared_error”*

The function to measure the quality of a split. Supported criteria are “squared_error” for the mean squared error, which is equal to variance reduction as feature selection criterion and minimizes the L2 loss using the mean of each terminal node, “friedman_mse”, which uses mean squared error with Friedman’s improvement score for potential splits, “absolute_error” for the mean absolute error, which minimizes the L1 loss using the median of each terminal node, and “poisson” which uses reduction in Poisson deviance to find splits. Training using “absolute_error” is significantly slower than when using “squared_error”.

! **Added in version 0.18:** Mean Absolute Error (MAE) criterion.

! **Added in version 1.0:** Poisson criterion.

max_depth : int, default=None

The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.

min_samples_split : int or float, default=2

The minimum number of samples required to split an internal node:

- If int, then consider `min_samples_split` as the minimum number.
- If float, then `min_samples_split` is a fraction and `ceil(min_samples_split * n_samples)` are the minimum number of samples for each split.

! **Changed in version 0.18:** Added float values for fractions.

min_samples_leaf : int or float, default=1

The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least `min_samples_leaf` training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression.

- If int, then consider `min_samples_leaf` as the minimum number.
- If float, then `min_samples_leaf` is a fraction and `ceil(min_samples_leaf * n_samples)` are the minimum number of samples for each node.

! **Changed in version 0.18:** Added float values for fractions.

min_weight_fraction_leaf : float, default=0.0

The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when `sample_weight` is not provided.

`max_features` : {"sqrt", "log2", None}, int or float, default=1.0

The number of features to consider when looking for the best split:

- If int, then consider `max_features` features at each split.
- If float, then `max_features` is a fraction and `max(1, int(max_features * n_features_in_))` features are considered at each split.
- If "sqrt", then `max_features=sqrt(n_features)` .
- If "log2", then `max_features=log2(n_features)` .
- If None or 1.0, then `max_features=n_features` .

Note

The default of 1.0 is equivalent to bagged trees and more randomness can be achieved by setting smaller values, e.g. 0.3.

Changed in version 1.1: The default of `max_features` changed from "auto" to 1.0.

Note: the search for a split does not stop until at least one valid partition of the node samples is found, even if it requires to effectively inspect more than `max_features` features.

`max_leaf_nodes` : int, default=None

Grow trees with `max_leaf_nodes` in best-first fashion. Best nodes are defined as relative reduction in impurity. If None then unlimited number of leaf nodes.

`min_impurity_decrease` : float, default=0.0

A node will be split if this split induces a decrease of the impurity greater than or equal to this value.

The weighted impurity decrease equation is the following:

$$N_t / N * (\text{impurity} - N_{t_R} / N_t * \text{right_impurity} - N_{t_L} / N_t * \text{left_impurity})$$

where N is the total number of samples, N_t is the number of samples at the current node, N_{t_L} is the number of samples in the left child, and N_{t_R} is the number of samples in the right child.

N , N_t , N_{t_R} and N_{t_L} all refer to the weighted sum, if `sample_weight` is passed.

! *Added in version 0.19.*

`bootstrap` : *bool, default=True*

Whether bootstrap samples are used when building trees. If False, the whole dataset is used to build each tree.

`oob_score` : *bool or callable, default=False*

Whether to use out-of-bag samples to estimate the generalization score. By default, `r2_score` is used. Provide a callable with signature `metric(y_true, y_pred)` to use a custom metric. Only available if `bootstrap=True`.

For an illustration of out-of-bag (OOB) error estimation, see the example [OOB Errors for Random Forests](#).

`n_jobs` : *int, default=None*

The number of jobs to run in parallel. `fit`, `predict`, `decision_path` and `apply` are all parallelized over the trees. `None` means 1 unless in a `joblib.parallel_backend` context. `-1` means using all processors. See [Glossary](#) for more details.

`random_state` : *int, RandomState instance or None, default=None*

Controls both the randomness of the bootstrapping of the samples used when building trees (if `bootstrap=True`) and the sampling of the features to consider when looking for the best split at each node (if `max_features < n_features`). See [Glossary](#) for details.

`verbose` : *int, default=0*

Controls the verbosity when fitting and predicting.

`warm_start` : *bool, default=False*

When set to `True`, reuse the solution of the previous call to fit and add more estimators to the ensemble, otherwise, just fit a whole new forest. See [Glossary](#) and [Fitting additional trees](#) for details.

`ccp_alpha` : *non-negative float, default=0.0*

Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than `ccp_alpha` will be chosen. By default, no

pruning is performed. See [Minimal Cost-Complexity Pruning](#) for details. See [Post pruning decision trees with cost complexity pruning](#) for an example of such pruning.

! Added in version 0.22.

max_samples : *int or float, default=None*

If bootstrap is True, the number of samples to draw from X to train each base estimator.

- If None (default), then draw `X.shape[0]` samples.
- If int, then draw `max_samples` samples.
- If float, then draw `max(round(n_samples * max_samples), 1)` samples. Thus, `max_samples` should be in the interval `(0.0, 1.0]`.

! Added in version 0.22.

monotonic_cst : *array-like of int of shape (n_features), default=None*

Indicates the monotonicity constraint to enforce on each feature.

- 1: monotonically increasing
- 0: no constraint
- -1: monotonically decreasing

If monotonic_cst is None, no constraints are applied.

Monotonicity constraints are not supported for:

- multioutput regressions (i.e. when `n_outputs_ > 1`),
- regressions trained on data with missing values.

Read more in the [User Guide](#).

! Added in version 1.4.

Attributes:

estimator_ : [*DecisionTreeRegressor*](#)

The child estimator template used to create the collection of fitted sub-estimators.

! Added in version 1.2: `base_estimator_` was renamed to `estimator_`.

estimators_ : *list of DecisionTreeRegressor*

The collection of fitted sub-estimators.

`feature_importances_` : *ndarray of shape (n_features,)*

The impurity-based feature importances.

`n_features_in_` : *int*

Number of features seen during [fit](#).

! Added in version 0.24.

`feature_names_in_` : *ndarray of shape (n_features_in_,)*

Names of features seen during [fit](#). Defined only when `x` has feature names that are all strings.

! Added in version 1.0.

`n_outputs_` : *int*

The number of outputs when `fit` is performed.

`oob_score_` : *float*

Score of the training dataset obtained using an out-of-bag estimate. This attribute exists only when `oob_score` is True.

`oob_prediction_` : *ndarray of shape (n_samples,) or (n_samples, n_outputs)*

Prediction computed with out-of-bag estimate on the training set. This attribute exists only when `oob_score` is True.

`estimators_samples_` : *list of arrays*

The subset of drawn samples for each base estimator.

 See also[`sklearn.tree.DecisionTreeRegressor`](#)

A decision tree regressor.

[`sklearn.ensemble.ExtraTreesRegressor`](#)

Ensemble of extremely randomized tree regressors.

[`sklearn.ensemble.HistGradientBoostingRegressor`](#)

A Histogram-based Gradient Boosting Regression Tree, very fast for big datasets ($n_{\text{samples}} \geq 10\,000$).

Notes

The default values for the parameters controlling the size of the trees (e.g. `max_depth`, `min_samples_leaf`, etc.) lead to fully grown and unpruned trees which can potentially be very large on some data sets. To reduce memory consumption, the complexity and size of the trees should be controlled by setting those parameter values.

The features are always randomly permuted at each split. Therefore, the best found split may vary, even with the same training data, `max_features=n_features` and `bootstrap=False`, if the improvement of the criterion is identical for several splits enumerated during the search of the best split. To obtain a deterministic behaviour during fitting, `random_state` has to be fixed.

The default value `max_features=1.0` uses `n_features` rather than `n_features / 3`. The latter was originally suggested in [1], whereas the former was more recently justified empirically in [2].

References

- [1] [L. Breiman, "Random Forests", Machine Learning, 45\(1\), 5-32, 2001.](#)
- [2] P. Geurts, D. Ernst., and L. Wehenkel, "Extremely randomized trees", Machine Learning, 63(1), 3-42, 2006.

Examples

```
>>> from sklearn.ensemble import RandomForestRegressor
>>> from sklearn.datasets import make_regression
>>> X, y = make_regression(n_features=4, n_informative=2,
...                       random_state=0, shuffle=False)
>>> regr = RandomForestRegressor(max_depth=2, random_state=0)
>>> regr.fit(X, y)
RandomForestRegressor(...)
>>> print(regr.predict([[0, 0, 0, 0]]))
[-8.32987858]
```

`apply(x)`

[\[source\]](#)

Apply trees in the forest to X, return leaf indices.

Parameters:

X : {array-like, sparse matrix} of shape (n_samples, n_features)

The input samples. Internally, its dtype will be converted to `dtype=np.float32`. If a sparse matrix is provided, it will be converted into a sparse `csr_matrix`.

Returns:

X_leaves : ndarray of shape (n_samples, n_estimators)

For each datapoint x in X and for each tree in the forest, return the index of the leaf x ends up in.

`decision_path(x)`

[\[source\]](#)

Return the decision path in the forest.

! Added in version 0.18.

Parameters:

X : {array-like, sparse matrix} of shape (n_samples, n_features)

The input samples. Internally, its dtype will be converted to `dtype=np.float32`. If a sparse matrix is provided, it will be converted into a sparse `csr_matrix`.

Returns:

indicator : sparse matrix of shape (n_samples, n_nodes)

Return a node indicator matrix where non zero elements indicates that the samples goes through the nodes. The matrix is of CSR format.

n_nodes_ptr : *ndarray of shape (n_estimators + 1,)*

The columns from `indicator[n_nodes_ptr[i]:n_nodes_ptr[i+1]]` gives the indicator value for the *i*-th estimator.

fit(*X*, *y*, *sample_weight=None*)

[\[source\]](#)

Build a forest of trees from the training set (*X*, *y*).

Parameters:

X : {array-like, sparse matrix} of shape (*n_samples*, *n_features*)

The training input samples. Internally, its dtype will be converted to `dtype=np.float32`. If a sparse matrix is provided, it will be converted into a sparse `csc_matrix`.

y : array-like of shape (*n_samples*,) or (*n_samples*, *n_outputs*)

The target values (class labels in classification, real numbers in regression).

sample_weight : array-like of shape (*n_samples*,), default=None

Sample weights. If None, then samples are equally weighted. Splits that would create child nodes with net zero or negative weight are ignored while searching for a split in each node. In the case of classification, splits are also ignored if they would result in any single class carrying a negative weight in either child node.

Returns:

self : object

Fitted estimator.

get_metadata_routing()

[\[source\]](#)

Get metadata routing of this object.

Please check [User Guide](#) on how the routing mechanism works.

Returns:

routing : *MetadataRequest*

A [MetadataRequest](#) encapsulating routing information.

get_params(*deep=True*)

[\[source\]](#)

Get parameters for this estimator.

Parameters:**deep** : *bool, default=True*

If True, will return the parameters for this estimator and contained subobjects that are estimators.

Returns:**params** : *dict*

Parameter names mapped to their values.

predict(*x*)[\[source\]](#)

Predict regression target for X.

The predicted regression target of an input sample is computed as the mean predicted regression targets of the trees in the forest.

Parameters:**X** : *{array-like, sparse matrix} of shape (n_samples, n_features)*

The input samples. Internally, its dtype will be converted to `dtype=np.float32`. If a sparse matrix is provided, it will be converted into a sparse `csr_matrix`.

Returns:**y** : *ndarray of shape (n_samples,) or (n_samples, n_outputs)*

The predicted values.

score(*X*, *y*, *sample_weight=None*)[\[source\]](#)

Return [coefficient of determination](#) on test data.

The coefficient of determination, R^2 , is defined as $(1 - \frac{u}{v})$, where u is the residual sum of squares `((y_true - y_pred)** 2).sum()` and v is the total sum of squares `((y_true - y_true.mean()) ** 2).sum()`. The best possible score is 1.0 and it can be negative (because the model can be arbitrarily worse). A constant model that always predicts the expected value of y , disregarding the input features, would get a R^2 score of 0.0.

Parameters:**X** : *array-like of shape (n_samples, n_features)*

Test samples. For some estimators this may be a precomputed kernel matrix or a list of generic objects instead with shape `(n_samples, n_samples_fitted)`, where `n_samples_fitted` is the number of samples used in the fitting for the estimator.

y : *array-like of shape (n_samples,) or (n_samples, n_outputs)*

True values for `X`.

sample_weight : *array-like of shape (n_samples,), default=None*

Sample weights.

Returns:

score : *float*

R^2 of `self.predict(X)` w.r.t. `y`.

Notes

The R^2 score used when calling `score` on a regressor uses `multioutput='uniform_average'` from version 0.23 to keep consistent with default value of `r2_score`. This influences the `score` method of all the multioutput regressors (except for [MultiOutputRegressor](#)).

`set_fit_request(*, sample_weight: bool / None / str = '$UNCHANGED$')` →

[RandomForestRegressor](#)

[\[source\]](#)

Configure whether metadata should be requested to be passed to the `fit` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a [meta-estimator](#) and metadata routing is enabled with `enable_metadata_routing=True` (see [sklearn.set_config](#)). Please check the [User Guide](#) on how the routing mechanism works.

The options for each parameter are:

- `True` : metadata is requested, and passed to `fit` if provided. The request is ignored if metadata is not provided.
- `False` : metadata is not requested and the meta-estimator will not pass it to `fit`.
- `None` : metadata is not requested, and the meta-estimator will raise an error if the user provides it.
- `str` : metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

! Added in version 1.3.

Parameters:

sample_weight : *str, True, False, or None,*
default=sklearn.utils.metadata_routing.UNCHANGED

Metadata routing for `sample_weight` parameter in `fit`.

Returns:

self : *object*

The updated object.

set_params(***params*)

[\[source\]](#)

Set the parameters of this estimator.

The method works on simple estimators as well as on nested objects (such as [Pipeline](#)). The latter have parameters of the form `<component>__<parameter>` so that it's possible to update each component of a nested object.

Parameters:

****params** : *dict*

Estimator parameters.

Returns:

self : *estimator instance*

Estimator instance.

set_score_request(*, *sample_weight*: [bool](#) / [None](#) / [str](#) = '\$UNCHANGED\$') →

[RandomForestRegressor](#)

[\[source\]](#)

Configure whether metadata should be requested to be passed to the `score` method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a [meta-estimator](#) and metadata routing is enabled with `enable_metadata_routing=True` (see [sklearn.set_config](#)). Please check the [User Guide](#) on how the routing mechanism works.

The options for each parameter are:

- `True` : metadata is requested, and passed to `score` if provided. The request is ignored if metadata is not provided.

- **False** : metadata is not requested and the meta-estimator will not pass it to **score** .
- **None** : metadata is not requested, and the meta-estimator will raise an error if the user provides it.
- **str** : metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (`sklearn.utils.metadata_routing.UNCHANGED`) retains the existing request. This allows you to change the request for some parameters and not others.

!

Added in version 1.3.

Parameters:

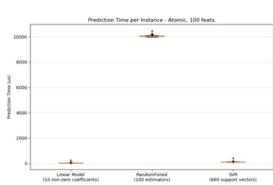
sample_weight : *str, True, False, or None,*
default=sklearn.utils.metadata_routing.UNCHANGED

Metadata routing for **sample_weight** parameter in **score** .

Returns:

self : *object*
The updated object.

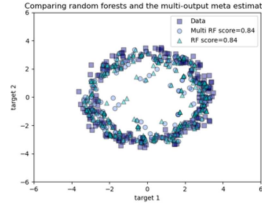
Gallery examples



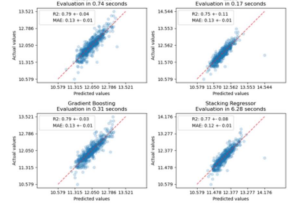
Prediction Latency



Comparing Random Forests and Histogram Gradient Boosting models



Comparing random forests and the multi-output meta estimator



Combine predictors using stacking

< Previous

[RandomForestClassifier](#)

Next >

[RandomTreesEmbedding](#)